

SYSTEM ARCHITECTURE

AI RAG Chatbot Data Pipelines & Storage Layers

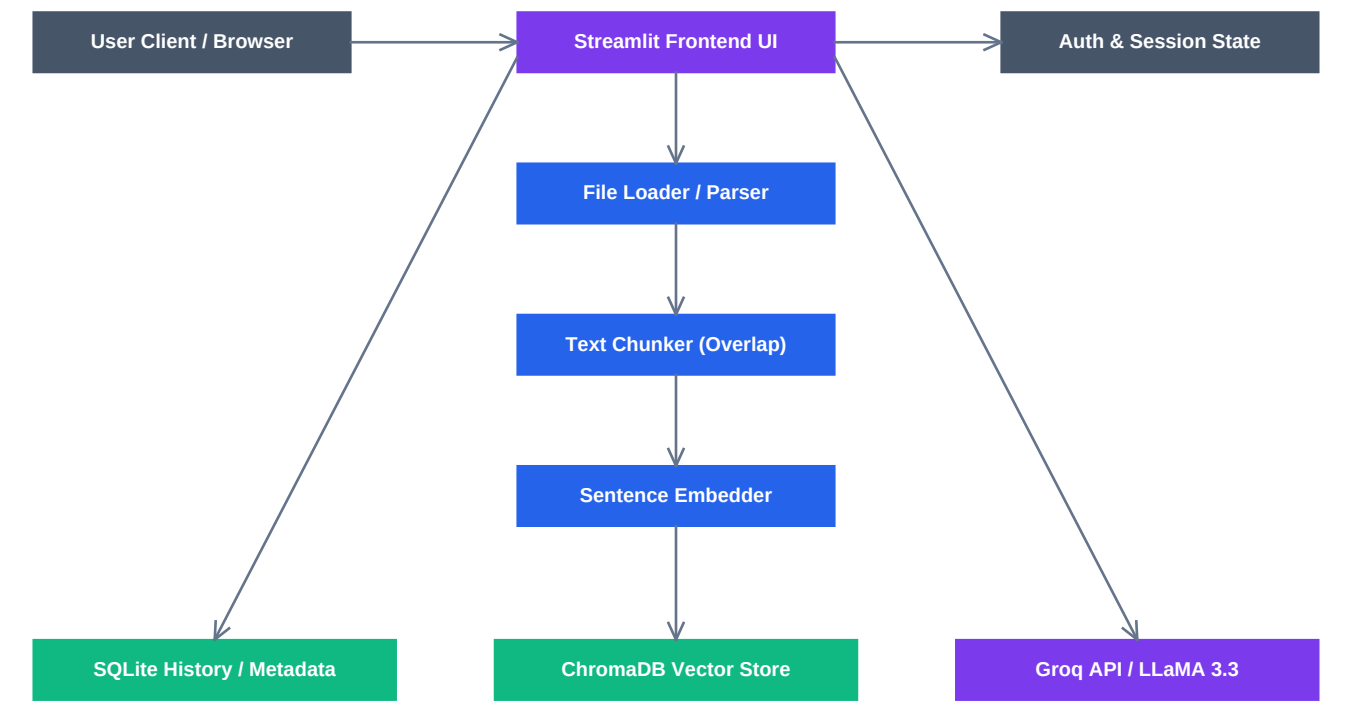
Technical System Design

Comprehensive mapping of data ingestion, indexing, and runtime RAG queries.

CORE BLUEPRINT FEATURES:

- Dynamic Multi-Format Parsing (PDF, DOCX, CSV, TXT, MD)
- In-Memory sentence-transformers Vector Generation
- Local Isolated ChromaDB Index Storage
- Groq LLaMA 3.3 70B Ultra-Fast Prompt Context Injection
- SQLite Credentials Verification & Message History Syncing

System Architecture Diagram



Component Operations Map:

Component	Technology	Primary Responsibility
Streamlit UI	Python Streamlit	Orchestrates chat inputs, layout cards, and multi-file uploads.
Parser / Loader	pdfplumber, docx	Reads and extracts texts from PDF, DOCX, CSV and TXT files.
Sentence Embedder	SentenceTransformers	Vectorizes word splits into 384-dimensional dense arrays.
ChromaDB Store	Local Vector DB	Performs cosine similarity lookups for relevant chunks.
SQLite Engine	SQLite3 Client	Houses credential hashes, document files registry, and message logs.
Groq Client	Groq SDK	Directs token streams, conversation pairs, and error fallback states.

Pipeline Execution Flow

1. Ingestion Pipeline (File Upload)

During document upload, file parser loaders (pdfplumber/python-docx/pandas) read raw binary structures. The string content is sent to the chunker to split it into 500-word segments (with 50-word margins). These text blocks are then converted to 384-d dense vectors by SentenceTransformer. The database helper logs files tracking UUIDs in SQLite, and ChromaDB indexes the vectors alongside reference sources.

2. Retrieval & Context Pipeline (Query Time)

When an query prompt is received, the message is stored in SQLite and vectorized. The query vector is compared against the user's document embeddings in ChromaDB. The top 4 most similar chunks (Cosine Similarity) are retrieved. These segments are structured inside the RAG context framework. The system client extracts conversational histories up to 20 messages, constructs the final prompt, and streams text generations from LLaMA 3.3 via Groq.

3. Security and Multi-User Isolation Model

Passwords are hashed using bcrypt. Access control is maintained using session states. Chat histories are filtered by username in SQLite. Documents are assigned unique UUIDs mapping to user accounts, ensuring ChromaDB retrieves records only belonging to the active user session.